

P

PARALELNI SISTEMI

1.

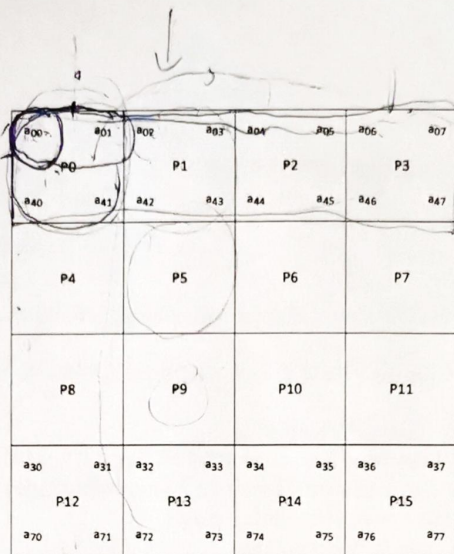
- a. Da li niti koje izvršavaju isti CUDA kernel mogu međusobno da komuniciraju? Kako? Da li postoje ograničenja?
- b. Da li blokovi niti koje izvršavaju isti CUDA kernel mogu međusobno da komuniciraju? Kako? Da li postoje ograničenja?
- c. Koja je razlika između deljive memorije i registara?
- d. Pretpostaviti da je neki CUDA kernel pokrenut sa 1000 blokova niti, svaki sa 512 niti. Ako je neka promenljiva deklarirana kao lokalna promenljiva u kernel funkciji, koliko verzija te promenljive će biti kreirano za vreme izvršavanja kernela?
- e. Potrebno je napisati CUDA kernel za sabiranje dva vektora, koji bi izraz bio najpogodniji za mapiranje indeksa niti i bloka na indeks rezultujućeg vektora:

- i. $i = \text{threadIdx.x} + \text{threadIdx.y};$
- ii. $i = \text{blockIdx.x} + \text{threadIdx.x};$
- iii. $i = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x};$
- iv. $i = \text{threadIdx.x} * \text{blockDim.x} + \text{blockIdx.x};$
- v. $i = \text{blockIdx.x} * \text{threadIdx.x};$

- f. Za prethodno navedeni kernel, neka je dužina vektora 2000, neka svaka nit računa samo jedan element rezultujućeg vektora, i neka je veličina bloka 512 niti. Koliko će ukupno niti biti u gridu?
- g. Napisati CUDA kernel za sabiranje dva vektora, i traženje srednje vrednosti elemenata rezultujućeg vektora. Obratiti pažnju na efikasnost paralelizacije. Omogućiti pozivanje kernela za proizvoljan broj blokova i niti.

2.

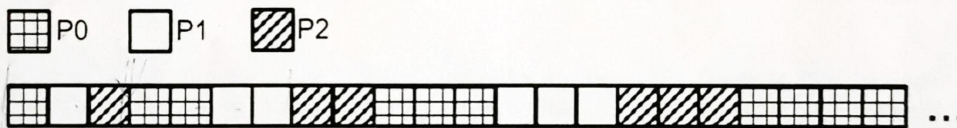
- a. Napisati MPI program koji realizuje množenje matrice $A_{k \times k}$ i vektora b_k , čime se dobija rezultujući vektor c_k . Matrica A i vektor b se inicijalizuju u master procesu. Broj procesa je p i uredeni su kao matrica $q \times q$ ($q^2 = p$). Matrica A je podeljena u blokove i master proces distribuira odgovarajuće blokove matrice A po procesima kao što je prikazano na slici 1. za $k=8$ i $p=16$. Vektor b je distribuiran po procesima tako da proces P_i dobija elemente sa indeksima $(i \% q) * s, (i \% q) * s + 1, \dots, (i \% q) * s + s - 1$ ($s = k/q$). Predvideti da se slanje vrednosti bloka matrice A svakom procesu obavlja odjednom. Svaki proces (uključujući i master) obavlja odgovarajuća izračunavanja i učestvuje u generisanju rezultata koji se prikazuje u procesu koji sadrži minimum svih vrednosti u matrici A. Predvideti da se slanje blokova matrice A svakom procesu obavlja sa po jednom naredbom MPI_Send kojom se šalje samo 1 izvedeni tip podatka. Slanje blokova vektora b i generisanje rezultata implementirati korišćenjem grupnih operacija i funkcija za kreiranje novih komunikatora.



b. Slanje blokova matrice A svakom procesu implementirati korišćenjem grupnih operacija i funkcija za kreiranje novih komunikatora

3. Napisati MPI program koji vrši paralelni upis i čitanje binarne datoteke, prema sledećim zahtevima:

- Svaki proces upisuje po $\frac{M(M+1)}{2}$ proizvoljnih celih brojeva u datoteku *file1.dat*. Upis se vrši upotrebom pojedinačnih pokazivača, dok redosled podataka u fajlu ide od podataka poslednjeg do podataka prvog procesa.
- Ponovo otvoriti datoteku. Svaki proces vrši čitanje upravo upisanih podataka upotrebom funkcija sa eksplicitnim pomerajem. Proveriti ispravnost pročitanih podataka.
- Upravo pročitane podatke upisati u novu datoteku, na način prikazan na slici (za slučaj od 3 procesa):



U poslednjem zahtevu posebno obratiti pažnju na efikasnost paralelizacije upisa. Broj M je celobrojna konstanta.

4. Napisati OpenMP kod koji sadrži sledeće ugnježdene petlje:

x = 0;

```
for (i = 2; i < N; i++)
  for (j = 0; j < M; j++)
  {
    x = x + a[i][j];
    b[i][j] = a[i-2][j];
  }
```

i proučiti da li je moguće izvršiti njihovu paralelizaciju. Ako nije, transformisati petlje tako da paralelizacija bude moguća. Nakon petlji treba prikazati vrednosti za promenljivu x, generisanu u okviru petlje. Testiranjem sekvencijalnog i paralelnog rešenja za proizvoljno N i M, i proizvoljan broj niti, pokazati korektnost paralelizovanog koda.